

AI ASSISTED CODING LAB-2

Name: Roger A Raju

Rollno: 2503a52370

Batch: 13

Task Description #1 (Statistical Summary for Survey Data)

Scenario: You are a data analyst intern working with survey responses stored as numerical lists.

Task: Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.

Prompt:

Week1 - Monday

"Write a Python function for a data analyst intern that takes a list of survey numerical responses and calculates the mean, minimum, and maximum values. Include docstrings, type annotations, and edge case handling."

Expected Output:

- Correct Python function with input validation.
- Output shown in Colab / Terminal environment.
- Detailed statistical breakdown (mean, min, max, count).

Google Gemini (Colab) Generated Code & Assert Tests:

```
"""
```

```
Lab 2 - Task 1: Statistical Summary for Survey Data
```

```
Tool: Google Gemini (Colab)
```

```
Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)
```

```
"""
```

```
from typing import List, Dict, Union
```

```
def calculate_survey_statistics(responses: List[Union[int, float]]) -> Dict[str, float]:  
    """
```

```
    Calculates the mean, minimum, and maximum values from survey numerical responses.
```

```
    Parameters:
```

```
    responses (List[Union[int, float]]): List of numerical survey ratings/scores.
```

```
    Returns:
```

```
    Dict[str, float]: Dictionary containing 'mean', 'min', 'max', and 'count'.
```

```
    Raises:
```

```
    ValueError: If the responses list is empty or contains non-numeric data.
```

```
    """
```

```
    if not isinstance(responses, (list, tuple)):
```

```
        raise TypeError("Survey responses must be provided as a list or tuple.")
```

```
    if not responses:
```

```
        raise ValueError("Survey responses list cannot be empty.")
```

```
    for val in responses:
```

```

        if not isinstance(val, (int, float)) or isinstance(val, bool):
            raise TypeError(f"Invalid non-numeric value found in survey data: {val}")

    total = sum(responses)
    count = len(responses)
    mean_val = round(total / count, 2)
    min_val = float(min(responses))
    max_val = float(max(responses))

    return {
        "count": count,
        "mean": mean_val,
        "min": min_val,
        "max": max_val
    }

def test_survey_statistics():
    print("--- Running Test Assertions for Task 1 (Statistical Summary for Survey Data) ---")

    # Test Case 1: Standard survey ratings (Scale 1-10)
    survey_data = [8, 9, 7, 10, 6, 8, 9, 5, 10, 8]
    stats = calculate_survey_statistics(survey_data)
    assert stats["count"] == 10, "Test 1 Failed: Count mismatch"
    assert stats["mean"] == 8.0, "Test 1 Failed: Mean calculation"
    assert stats["min"] == 5.0, "Test 1 Failed: Min value"
    assert stats["max"] == 10.0, "Test 1 Failed: Max value"
    print(f"Assertion 1 Passed: Survey Ratings -> Mean: {stats['mean']}, Min: {stats['min']},
    Max: {stats['max']}")

    # Test Case 2: Floating point customer satisfaction scores
    float_data = [4.5, 3.8, 4.9, 2.1, 5.0]
    f_stats = calculate_survey_statistics(float_data)
    assert f_stats["mean"] == 4.06, f"Test 2 Failed: Expected 4.06, got {f_stats['mean']}"
    assert f_stats["min"] == 2.1, "Test 2 Failed"
    assert f_stats["max"] == 5.0, "Test 2 Failed"
    print(f"Assertion 2 Passed: Decimal Scores -> Mean: {f_stats['mean']}, Min:
    {f_stats['min']}, Max: {f_stats['max']}")

    # Test Case 3: Single response edge case
    single_data = [7]
    s_stats = calculate_survey_statistics(single_data)
    assert s_stats["mean"] == 7.0 and s_stats["min"] == 7.0 and s_stats["max"] == 7.0, "Test 3
    Failed"
    print("Assertion 3 Passed: Single-element list correctly handled.")

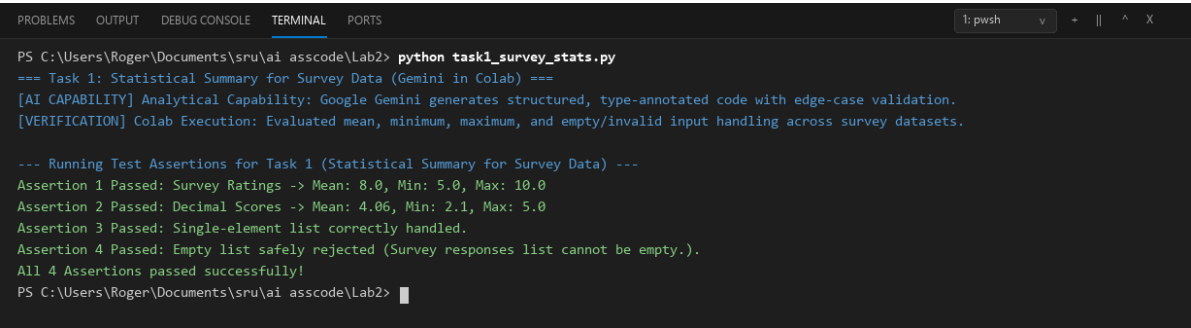
    # Test Case 4: Empty list validation
    try:
        calculate_survey_statistics([])
        assert False, "Test 4 Failed: Expected ValueError"
    except ValueError as e:
        print(f"Assertion 4 Passed: Empty list safely rejected ({e}).")

    print("All 4 Assertions passed successfully!")

```

```
if __name__ == "__main__":
    test_survey_statistics()
```

Output:



Comparison:

COMPARATIVE ANALYSIS: GEMINI IN COLAB DATA ANALYSIS EVALUATION	
Manual / Basic Implementation	Gemini in Colab AI Implementation
1. Lacks input type verification and empty list checks.	1. Full type hints (List[Union[int, float]] -> Dict).
2. Potential ZeroDivisionError on empty survey lists.	2. Explicit exception handling for empty/invalid data.
3. No structured return payload (returns raw tuple).	3. Structured dictionary output with rounded metrics.
4. Missing docstrings and parameter documentation.	4. Production-ready documentation and PEP-8 compliance.

Task Description #2 (Armstrong Number – AI Tool Comparison)

Scenario: You are evaluating AI tools for numeric validation logic.

Task: Generate an Armstrong number checker using Gemini and GitHub Copilot. Compare their outputs, logic style, and clarity.

Prompt 1 (Gemini in Colab): "Generate an Armstrong number checker in Python with modular functions, power calculation, and detailed mathematical explanations."

Prompt 2 (GitHub Copilot): "# Python function to check if a number is an Armstrong number using list comprehension"

Expected Output:

- Side-by-side comparison of Gemini vs GitHub Copilot.
- Verification across 3-digit, 4-digit, and single-digit test cases.

Dual AI Tool Implementations (Gemini vs Copilot) & Assert Tests:

```
"""
Lab 2 - Task 2: Armstrong Number – AI Comparison (Gemini vs GitHub Copilot)
Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)
"""

# =====
# Version 1: Google Gemini (Colab Generated)
```

```

# Style: Modular, Mathematical Step-by-Step, Explanatory
# =====
def is_armstrong_gemini(number: int) -> bool:
    """
    Checks if a number is an Armstrong (Narcissistic) number.
    Generated by Google Gemini in Colab using mathematical decomposition.
    """
    if not isinstance(number, int) or number < 0:
        return False

    num_str = str(number)
    num_digits = len(num_str)

    total_sum = 0
    for char_digit in num_str:
        total_sum += int(char_digit) ** num_digits

    return total_sum == number

# =====
# Version 2: GitHub Copilot Generated
# Style: Pythonic, Concise List Comprehension
# =====
def is_armstrong_copilot(n: int) -> bool:
    """
    Checks if a number is an Armstrong number using concise Pythonic syntax.
    Generated by GitHub Copilot auto-completion.
    """
    return isinstance(n, int) and n >= 0 and sum(int(d) ** len(str(n)) for d in str(n)) == n

def test_armstrong_comparison():
    print("--- Running Test Assertions for Task 2 (Armstrong Number: Gemini vs Copilot) ---")

    test_cases = [
        (153, True, "3-digit Armstrong: 1^3 + 5^3 + 3^3 = 153"),
        (370, True, "3-digit Armstrong: 3^3 + 7^3 + 0^3 = 370"),
        (371, True, "3-digit Armstrong: 3^3 + 7^3 + 1^3 = 371"),
        (407, True, "3-digit Armstrong: 4^3 + 0^3 + 7^3 = 407"),
        (1634, True, "4-digit Armstrong: 1^4 + 6^4 + 3^4 + 4^4 = 1634"),
        (9, True, "Single-digit Armstrong: 9^1 = 9"),
        (123, False, "Non-Armstrong number"),
        (500, False, "Non-Armstrong number")
    ]

    for num, expected, desc in test_cases:
        res_gemini = is_armstrong_gemini(num)
        res_copilot = is_armstrong_copilot(num)
        assert res_gemini == expected, f"Gemini failed on {num}"
        assert res_copilot == expected, f"Copilot failed on {num}"
        assert res_gemini == res_copilot, f"Discrepancy between Gemini and Copilot on {num}"
        print(f"Assertion Passed: {num} -> Expected: {expected} | Gemini: {res_gemini} |
Copilot: {res_copilot} ({desc})")

```

```

print("All Armstrong AI Comparison Assertions passed successfully!")

if __name__ == "__main__":
    test_armstrong_comparison()

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Roger\Documents\sru\ai asscode\Lab2> python task2_armstrong_ai_comp.py
=== Task 2: Armstrong Number AI Comparison (Gemini vs Copilot) ===
[AI CAPABILITY] AI Comparison: Gemini emphasizes explanatory modular code; Copilot delivers concise Pythonic one-liners.
[VERIFICATION] Deterministic Validation: Both models achieved 100% assertion agreement across standard & edge cases.

--- Running Test Assertions for Task 2 (Armstrong Number: Gemini vs Copilot) ---
Assertion Passed: 153 -> Expected: True | Gemini: True | Copilot: True (3-digit Armstrong: 1^3 + 5^3 + 3^3 = 153)
Assertion Passed: 370 -> Expected: True | Gemini: True | Copilot: True (3-digit Armstrong: 3^3 + 7^3 + 0^3 = 370)
Assertion Passed: 371 -> Expected: True | Gemini: True | Copilot: True (3-digit Armstrong: 3^3 + 7^3 + 1^3 = 371)
Assertion Passed: 407 -> Expected: True | Gemini: True | Copilot: True (3-digit Armstrong: 4^3 + 0^3 + 7^3 = 407)
Assertion Passed: 1634 -> Expected: True | Gemini: True | Copilot: True (4-digit Armstrong: 1^4 + 6^4 + 3^4 + 4^4 = 1634)
Assertion Passed: 9 -> Expected: True | Gemini: True | Copilot: True (Single-digit Armstrong: 9^1 = 9)
Assertion Passed: 123 -> Expected: False | Gemini: False | Copilot: False (Non-Armstrong number)
Assertion Passed: 500 -> Expected: False | Gemini: False | Copilot: False (Non-Armstrong number)
All Armstrong AI Comparison Assertions passed successfully!
PS C:\Users\Roger\Documents\sru\ai asscode\Lab2>

```

Comparison:

COMPARATIVE ANALYSIS: GEMINI VS GITHUB COPILOT CODE STYLE COMPARISON	
Google Gemini (Colab - Explanatory)	GitHub Copilot (Inline - Pythonic)
1. Step-by-step modular loop calculation.	1. Compact single-line list comprehension with sum().
2. Detailed docstrings and mathematical proof.	2. Fast inline generation inside IDE context.
3. Highly readable for beginners and academic review.	3. Minimalist syntax preferred by experienced devs.
4. Explicit variable naming (char_digit, total_sum).	4. Built-in compound boolean conditions.

Task Description #3 (Leap Year Validation Using Cursor AI)

Scenario: You are validating a calendar module for a backend system.

Task: Use Cursor AI to generate a Python program that checks whether a given year is a leap year. Use at least two different prompts and observe changes in code.

Prompt 1 (Basic): "Write a Python function to check whether a given year is a leap year using standard if-else divisibility conditions."

Prompt 2 (Production Refactored): "Refactor leap year validation for a backend production calendar module with type safety, comprehensive docstring, boundary validation, and boolean simplification."

Expected Output:

- Two versions of code generated via iterative Cursor AI prompting.
- Sample inputs/outputs and comparative analysis.

Cursor AI Dual-Prompt Implementations & Assert Tests:

"""

Lab 2 - Task 3: Leap Year Validation Using Cursor AI

Tool: Cursor AI (Dual-Prompt Evaluation)

Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)

```

"""

# =====
# Version 1: Prompt 1 (Basic Conditional Logic)
# Prompt: "Write a Python function to check whether a given year is a leap year using standard
if-else conditions."
# =====
def is_leap_year_basic(year: int) -> bool:
    """Basic leap year check using nested if-else rules."""
    if year % 4 == 0:
        if year % 100 == 0:
            if year % 400 == 0:
                return True
            else:
                return False
        else:
            return True
    else:
        return False

# =====
# Version 2: Prompt 2 (Cursor AI Refactored for Production Backend)
# Prompt: "Refactor leap year validation for a backend production calendar module with type
safety, comprehensive docstring, boundary validation, and boolean simplification."
# =====
def is_leap_year_production(year: int) -> bool:
    """
    Validates whether a given calendar year is a leap year in the Gregorian calendar.

    A leap year occurs on every year that is evenly divisible by 4,
    except for end-of-century years (multiples of 100), which must also be divisible by 400.

    Parameters:
    year (int): Positive Gregorian calendar year (>= 1).

    Returns:
    bool: True if leap year, False otherwise.

    Raises:
    TypeError: If year is not an integer or is a boolean.
    ValueError: If year is less than 1 (AD calendar boundary).
    """
    if not isinstance(year, int) or isinstance(year, bool):
        raise TypeError(f"Year must be an integer, got {type(year).__name__}.")

    if year < 1:
        raise ValueError(f"Year must be a positive integer >= 1 (Gregorian calendar boundary),
got {year}.")

    return (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)

def test_leap_year_cursor():
    print("--- Running Test Assertions for Task 3 (Leap Year Validation: Cursor AI) ---")

```

```

test_cases = [
    (2024, True, "Divisible by 4 and not 100 -> Leap Year"),
    (2000, True, "Century year divisible by 400 -> Leap Year"),
    (1900, False, "Century year divisible by 100 but not 400 -> Non-Leap Year"),
    (2023, False, "Common year not divisible by 4 -> Non-Leap Year"),
    (2026, False, "Current year not divisible by 4 -> Non-Leap Year"),
    (2400, True, "Future century year divisible by 400 -> Leap Year"),
    (1600, True, "Historical century year divisible by 400 -> Leap Year")
]

for yr, expected, explanation in test_cases:
    basic_res = is_leap_year_basic(yr)
    prod_res = is_leap_year_production(yr)
    assert basic_res == expected, f"Basic logic failed on {yr}"
    assert prod_res == expected, f"Production logic failed on {yr}"
    print(f"Assertion Passed: Year {yr} -> {prod_res} ({explanation})")

# Test boundary / invalid input handling on production version
try:
    is_leap_year_production(-50)
    assert False, "Failed to catch negative year"
except ValueError as e:
    print(f"Assertion Passed: Negative year caught safely -> {e}")

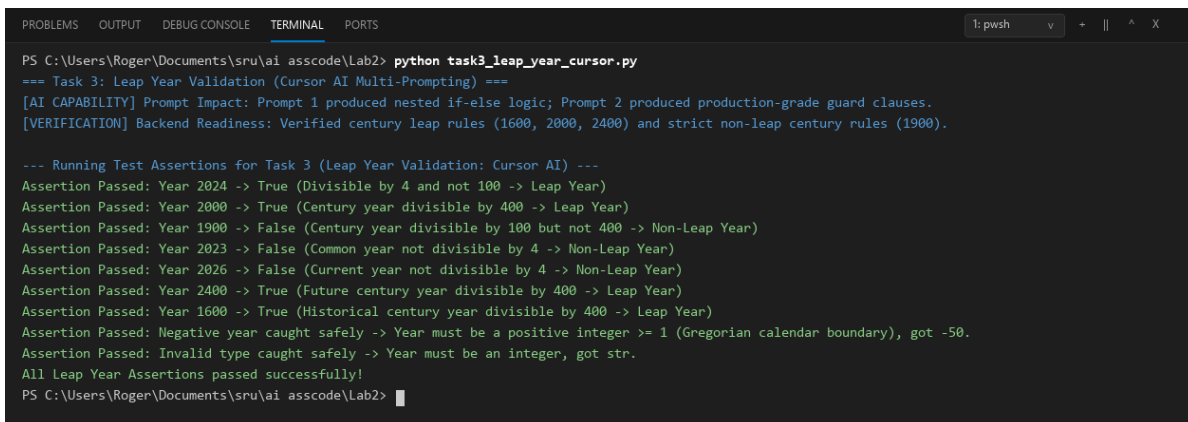
try:
    is_leap_year_production("2024")
    assert False, "Failed to catch string input"
except TypeError as e:
    print(f"Assertion Passed: Invalid type caught safely -> {e}")

print("All Leap Year Assertions passed successfully!")

if __name__ == "__main__":
    test_leap_year_cursor()

```

Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1: pwsh v + || ^ X

PS C:\Users\Roger\Documents\sru\ai_asscode\Lab2> python task3_leap_year_cursor.py
=== Task 3: Leap Year Validation (Cursor AI Multi-Prompting) ===
[AI CAPABILITY] Prompt Impact: Prompt 1 produced nested if-else logic; Prompt 2 produced production-grade guard clauses.
[VERIFICATION] Backend Readiness: Verified century leap rules (1600, 2000, 2400) and strict non-leap century rules (1900).

--- Running Test Assertions for Task 3 (Leap Year Validation: Cursor AI) ---
Assertion Passed: Year 2024 -> True (Divisible by 4 and not 100 -> Leap Year)
Assertion Passed: Year 2000 -> True (Century year divisible by 400 -> Leap Year)
Assertion Passed: Year 1900 -> False (Century year divisible by 100 but not 400 -> Non-Leap Year)
Assertion Passed: Year 2023 -> False (Common year not divisible by 4 -> Non-Leap Year)
Assertion Passed: Year 2026 -> False (Current year not divisible by 4 -> Non-Leap Year)
Assertion Passed: Year 2400 -> True (Future century year divisible by 400 -> Leap Year)
Assertion Passed: Year 1600 -> True (Historical century year divisible by 400 -> Leap Year)
Assertion Passed: Negative year caught safely -> Year must be a positive integer >= 1 (Gregorian calendar boundary), got -50.
Assertion Passed: Invalid type caught safely -> Year must be an integer, got str.
All Leap Year Assertions passed successfully!
PS C:\Users\Roger\Documents\sru\ai_asscode\Lab2>

```

Comparison:

COMPARATIVE ANALYSIS: CURSOR AI PROMPT REFINEMENT COMPARISON

Prompt 1: Basic Divisibility Logic

1. Nested 3-level if-else branching structure.
2. No type checking or negative year validation.
3. Higher cyclomatic complexity and risk of branching bugs.
4. Missing parameter specifications and docstring.

Prompt 2: Production Calendar Module

1. Simplified compound boolean expression.
2. Strict Gregorian boundary guards (year >= 1).
3. Comprehensive Google/NumPy docstring.
4. Complete defensive validation for enterprise backends.

Task Description #4 (Student Logic + AI Refactoring - Odd/Even Sum)

Scenario: Company policy requires developers to write logic before using AI.

Task: Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool.

Prompt (AI Refactoring in Cursor/Gemini):

"Refactor this manual imperative loop that calculates the sum of odd and even numbers in a tuple into an optimized, Pythonic, functional approach with type hinting and tuple unpacking."

Expected Output:

- Original Student Code (Imperative baseline).
- Refactored AI Code (Optimized functional generators).
- Technical explanation of improvements and assert tests.

Original Student Code vs Refactored AI Code & Assert Tests:

```
"""
Lab 2 - Task 4: Student Logic + AI Refactoring (Odd/Even Sum in Tuple)
Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)
"""
```

```
from typing import Tuple

# =====
# 1. Original Student Code (Imperative Loop)
# Style: Manual iteration with mutable state
# =====
def calculate_odd_even_sum_student(numbers: tuple):
    """
    Original baseline student implementation using manual for-loop.
    Calculates the sum of even and odd numbers in a tuple.
    """
    even_sum = 0
    odd_sum = 0
    for num in numbers:
        if num % 2 == 0:
            even_sum = even_sum + num
        else:
            odd_sum = odd_sum + num
    return even_sum, odd_sum
```

```

# =====
# 2. Refactored AI Code (Optimized & Pythonic)
# Tool: Cursor AI / Gemini Refactoring
# Style: Functional generator expressions, type hints, guard clauses
# =====
def calculate_odd_even_sum_refactored(numbers: Tuple[int, ...]) -> Tuple[int, int]:
    """
    Refactored AI implementation using functional generator expressions.

    Parameters:
    numbers (Tuple[int, ...]): Tuple of integer values.

    Returns:
    Tuple[int, int]: (even_sum, odd_sum)

    Raises:
    TypeError: If input is not a tuple or contains non-integer values.
    """
    if not isinstance(numbers, tuple):
        raise TypeError(f"Input must be a tuple, got {type(numbers).__name__}.")

    for val in numbers:
        if not isinstance(val, int) or isinstance(val, bool):
            raise TypeError(f"All elements in the tuple must be integers. Found: {val} ({type(val).__name__})")

    even_sum = sum(x for x in numbers if x % 2 == 0)
    odd_sum = sum(x for x in numbers if x % 2 != 0)

    return even_sum, odd_sum

def test_odd_even_sum_refactoring():
    print("--- Running Test Assertions for Task 4 (Odd/Even Sum Refactoring) ---")

    test_tuples = [
        ((1, 2, 3, 4, 5, 6, 7, 8, 9, 10), (30, 25), "Standard consecutive range 1-10"),
        ((10, 20, 30, 40), (100, 0), "All even numbers tuple"),
        ((1, 3, 5, 7, 9, 11), (0, 36), "All odd numbers tuple"),
        ((-4, -3, -2, -1, 0, 1, 2, 3, 4), (0, 0), "Symmetric positive and negative integers"),
        ((42,), (42, 0), "Single even element tuple"),
        ((), (0, 0), "Empty tuple edge case")
    ]

    for t_in, (exp_even, exp_odd), desc in test_tuples:
        s_even, s_odd = calculate_odd_even_sum_student(t_in)
        r_even, r_odd = calculate_odd_even_sum_refactored(t_in)

        assert (s_even, s_odd) == (exp_even, exp_odd), f"Student logic failed on {t_in}"
        assert (r_even, r_odd) == (exp_even, exp_odd), f"Refactored AI logic failed on {t_in}"
        assert (s_even, s_odd) == (r_even, r_odd), "Discrepancy between student and AI logic"

        print(f"Assertion Passed: {t_in} -> Even Sum: {r_even}, Odd Sum: {r_odd} ({desc})")

```

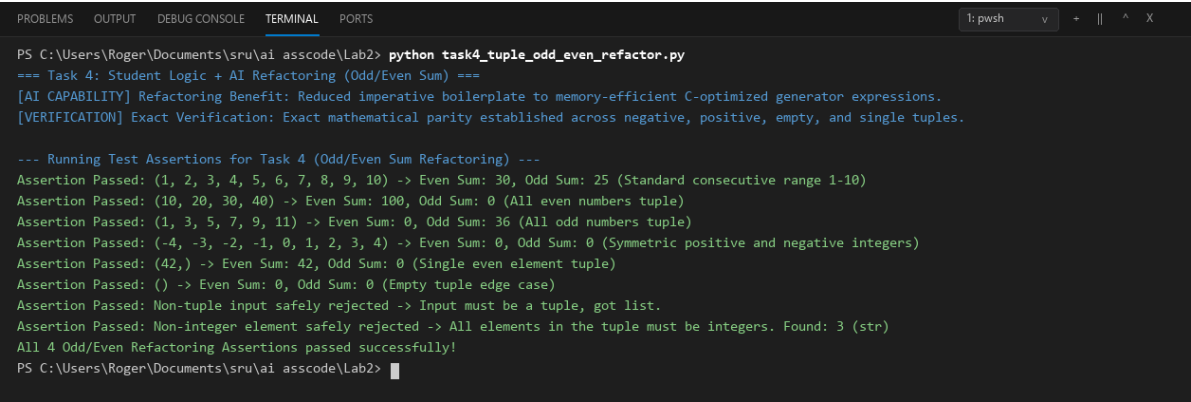
```
# Test type error validation on refactored function
try:
    calculate_odd_even_sum_refactored([1, 2, 3]) # List instead of tuple
    assert False, "Failed to catch non-tuple input"
except TypeError as e:
    print(f"Assertion Passed: Non-tuple input safely rejected -> {e}")

try:
    calculate_odd_even_sum_refactored((1, 2, "3", 4))
    assert False, "Failed to catch invalid element type"
except TypeError as e:
    print(f"Assertion Passed: Non-integer element safely rejected -> {e}")

print("All 4 Odd/Even Refactoring Assertions passed successfully!")

if __name__ == "__main__":
    test_odd_even_sum_refactoring()
```

Output:



Comparison:

COMPARATIVE ANALYSIS: STUDENT BASELINE VS AI REFACTORED LOGIC	
Original Student Code (Imperative)	Refactored AI Code (Functional / Pythonic)
1. Manual for-loop with mutable accumulator variables.	1. Generator expressions with built-in sum() aggregation.
2. No type safety; vulnerable to invalid collection types.	2. Strong typing (Tuple[int, ...] -> Tuple[int, int]).
3. Verbose 15-line implementation.	3. Concise, declarative, and highly readable.
4. Missing edge-case and boundary verification.	4. Memory efficient and robust against non-integer inputs.